MODULE-5

INTRODUCTION

Quality:

- Quality is generally agreed to be **'a good thing'.**
- In practice what people really mean by the '**quality' of a system can be vague, undefinedattribute.**

Objective assessment:

- We need to judge objectively whether a system meets our quality requirements and this need measurement. It is Critical for package selection. For example.
  - ✓ Delivering a high-quality product is one of the major objectives of all organizations. Traditionally, the quality of a product means how much it fits into its specified purpose. A productis of ***good quality if it performs according to the user's requirements***.
  - ✓ Good quality software should meetall objectives defined in the SRS document. It is the responsibility of the quality managers to **ensure that** the software attains the required level of quality.
- Waiting until the system is complete to measure quality is **too late.During development**, it's important to:
  - ✓ **Assess the likely quality of the final system.**
  - ✓ **Ensure development methods** will produce the desired quality.Potential customers, like Amanda at IOE, might focus on:
    - ➢ Checking if **suppliers use the best development methods**.
    - ➢ Ensuring these methods will **lead to the required quality in the final system.**

THE PLACE OF SOFTWARE QUALITY IN PROJECT PLANNING

- Quality will be of concern at all stages of project planning and execution but will be of particular interest to the Stepwise framework.

1. **Step 1: Identifying project scope and objectives.: -** Some objectives could relate to the quality of theapplication to be delivered.
2. **Step 2: Identifying project infrastructure: -** Within this step activity 2. involves identifying installationstandards and procedures. Some of these will almost certainly be about quality requirements.
3. **Step 3: Analyze project characteristics: - In** this activity the application to be implemented will beexamined to see if it has any special quality requirements.

   Example: - Safety-critical applications might require additional activities such as n-version development, where multiple teams develop versions of the same software to cross-check outputs.
4. **Step 4: Identify the product and activities of the project: - It** is at that point that the entry, exit and process requirements are identified for each activity.
5. **Step 5: Review and publicize plan: -** At this stage the overall quality aspects of the project plan are reviewed.

VTUSYNC.IN

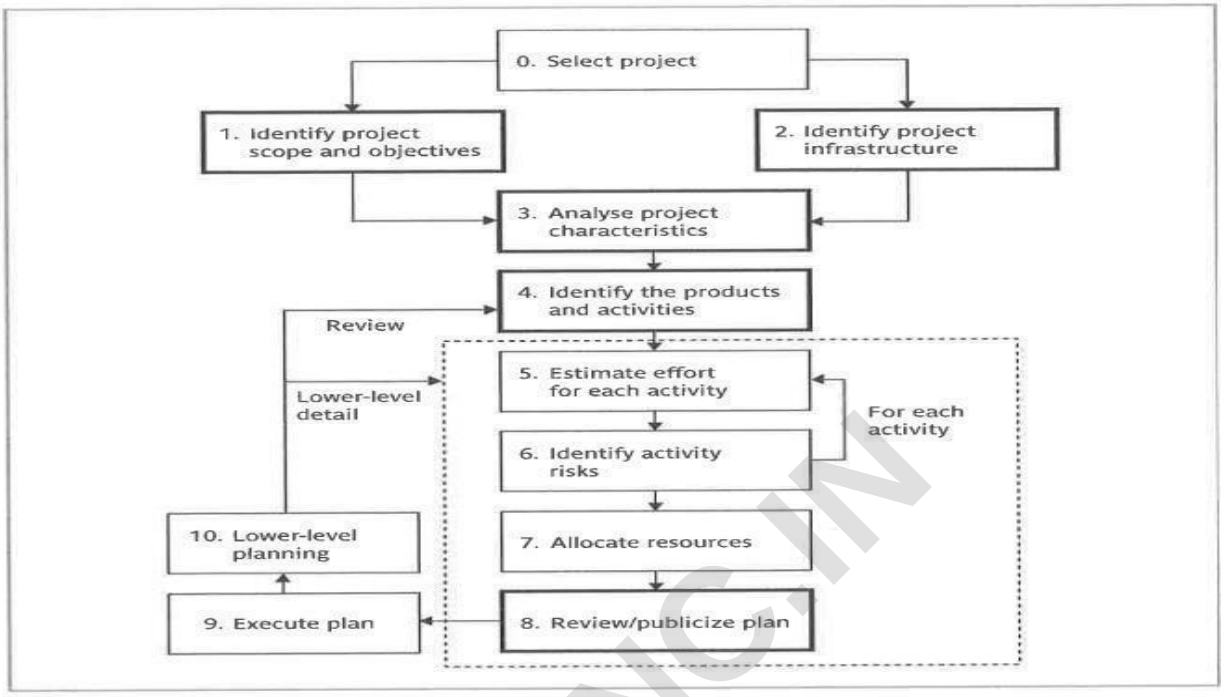


FIGURE 13.1 The place of software quality in Step Wise

VTUSYNC.IN

THE IMPORTANCE OF SOFTWARE QUALITY

- **Quality is the important aspect of all organizations.** Good quality software is the requirement of all users. There are so many reasons that describe why the quality of software is important;few among of those which are most important are described below:

Increasingly criticality of software:

- The final customer or user is naturally anxious about the general quality of software, especially about reliability.
- They are concerned about safety because of their dependency on software systems such as aircraft control systems are more safety critical systems.

Earlier detection of errors during development-

- As software is developed through several phases; **output of one phase is given as input to the other one. So, if an error in the initial phase is not found, then at the later stage, it is difficult to fix that error, and the cost indulged is more.**

The intangibility of software:

- Difficulty in verifying the satisfactory completion of project tasks.
- Tangibility is achieved by requiring developers to produce "deliverables" that can be examined for quality.

Accumulating errors during software development:

- Errors in earlier steps can propagate and accumulate in later steps.
- Errors found later in the project are more expensive to fix.
- The unknown number of errors makes the debugging phase difficult to control.

DEFINING SOFTWARE QUALITY

- Quality is a rather vague term, and we need to define carefully what we mean by it. For any software system there should be three specifications.
  - ✓ **A functional specification** describing what the system is to do
  - ✓ A **quality specification** concerned with how well the function are to operate
  - ✓ A **resource specification** concerned with how much is to be spent on the system.
- Attempt to identify specific product qualities that are appropriate to software, for instance, grouped software qualities **into three sets. Product operation qualities, Product revision qualities and product transition qualities.**

Product operation qualities:

- **Correctness**: The extent to which a program satisfies its specification and fulfils user objective.
- **Reliability**: The extent to which a program can be expected to perform its intended function with requiredprecision.
- **Efficiency**: The amount of computer resources required by software.
- **Integrity**: Extent to which access to software or data by unauthorized persons can be controlled.
- **Usability**: The effort required to learn, operate, prepare input and interpret output.

Product Revision Qualities:

- **Maintainability**: The effort required to locate and fix an error in an operational program
- **Testability**: The effort required to test a program to ensure it performs its intended function.
- **Flexibility**: The effort required to modify an operational program.

Product Transition Qualities:

Portability: The efforts require to transfer a program from one hardware configuration and or software system environment to another.

Reusability: The extent to which a program can be used in other applications.

Interoperability: The efforts required to couple one system to another. When there is concern about the need for a specific quality characteristic in a software product then aquality specification with the following minimum details should be drafted.

Definition/Description

- **Definition:** Clear definition of the quality characteristic.
- **Description:** Detailed description of what the quality characteristic entails.

Scale Unit of Measurement:

- The unit used to measure the quality characteristic (e.g., faults per thousand lines of code).

Test

- Practical Test: The process used to test the extent to which the quality attribute exists.

Minimally Acceptable

- **Worst Acceptable Value**: The lowest acceptable value, below which the product would berejected.

VTUSYNC.IN

Target Range

- **Planned Range**: The range of values within which it is planned that the quality measurement value should lie.

Current Value

- **Now**: The value that applies currently to the quality characteristic.

Measurements Applicable to Quality Characteristics in Software

- When assessing quality characteristics in software, multiple measurements may be applicable. E xample, in the case of reliability, measurements could include:

Availability:

Definition: Percentage of a particular time interval that a system is usable.
Scale: Percentage (%).
Test: Measure the system's uptime versus downtime over a specified period.

- **Minimally Acceptable:** Typically, high availability is desirable, specifics depend on system requirements.
- **Target Range: E.g., 99.9% uptime.**

Mean Time Between Failures (MTBF):

Definition: Total service time divided by the number of failures.
Scale: Time (e.g., hours, days).
Test: Calculate the average time elapsed between system failures.

- **Minimally Acceptable:** Longer MTBF indicates higher reliability, minimum varies by systemcriticality.
- **Target Range:** E.g., MTBF of 10,000 hours.

Failure on Demand:

Definition: Probability that a system will not be available when required, or probability that atransaction will fail.
Scale: Probability (0 to 1).
Test: Evaluate the system's response to demand or transaction processing.

- **Minimally Acceptable:** Lower probability of failure is desired; varies by system criticality.
- **Target Range:** E.g., Failure on demand probability of less than 0.01.

Support Activity:

Definition: Number of fault reports generated and processed.
Scale: Count (number of reports).
Test: Track and analyze the volume and resolution time of fault reports.

- **Minimally Acceptable**: Lower number of fault reports indicates better reliability.
- **Target Range**: E.g., Less than 10 fault reports per month.

Maintainability and Related Qualities:

- **Maintainability:** How quickly a fault can be corrected once detected.
- **Changeability**: Ease with which software can be modified.
- **Analyzability**: Ease with which causes of failure can be identified, contributing to maintainability. These measurements help quantify and assess the reliability and

VTUSYNC.IN

maintainability of software systems, ensuring they meet desired quality standards.

SOFTWARE QUALITY MODELS

- ❖ It is hard to directly measure the quality of a software. It can be expressed in terms of several attributes of a software that can be directly measured.
- ❖ The quality models give a characterization (hierarchical) of software quality in terms of a set of characteristics of the software. The bottom level of the hierarchical can be directly measured, thereby enabling a quantitative assessment of the quality of the software.

There are several well-established quality models.

1. ***McCall's model.***
2. ***Dromey's Model.***
3. ***Boehm's Model.***
4. ***ISO 9126 Model.***

Garvin's Quality Dimensions: David Gravin, a professor at Harvard Business school, defined thequality of any product in terms of eight general attributes of the product.

- ➢ **Performance:** How well if performs the job.
- ➢ **Features**: How well it supports the required features.
- ➢ **Reliability**: Probability of a product working satisfactorily within a specified period.
- ➢ **Conformance:** Degree to which the product meets the requirements.
- ➢ **Durability:** Measure of the product life.
- ➢ **Serviceability**: Speed and effectiveness maintenance.
- ➢ **Aesthetics:** The look and feel of the product.
- ➢ **Perceived quality**: User's opinion about the product quality.

1) McCall' Model: McCall defined the quality of a software in terms of **three broad parameters:** its operational characteristics, how easy it is to fix defects and how easy it is to part it to different platforms is as shown below Figure. These three high-level quality attributes are defined based on the following **eleven attributes of the software**:

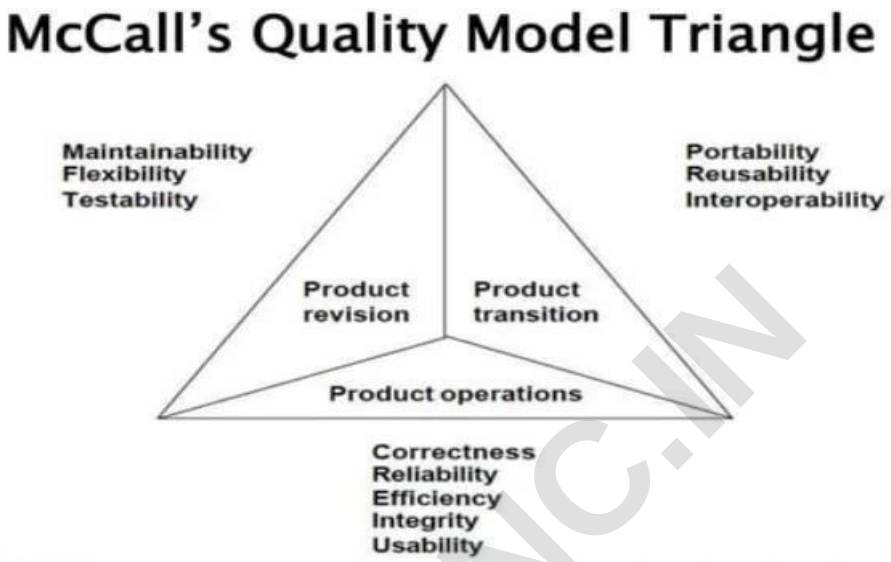
i. **Correctness:** The extent to which a software product satisfies its specifications.
ii. **Reliability:** The probability of the software product working satisfactorily over a given duration.
iii. **Efficiency:** The amount of computing resources required to perform the required functions.
iv. **Integrity:** The extent to which the data of the software product remains valid.
v. **Usability:** The effort required to operate the software product.
vi. **Maintainability:** The ease with which it is possible to locate and fix bugs in the software product.
vii. **Flexibility:** The effort required to adapt the software product to changing requirements.
viii. **Testability:** The effort required to test a software product to ensure that it performs its intended function.
ix. **Portability:** The effort required to transfer the software product from one hardware or

VTUSYNC.IN

software system environment to another.

x. **Reusability:** The extent to which a software can be reused in other applications.

xi. **Interoperability:** The effort required to integrate the software with other software.



2) Dromey's model: Dromey proposed that software product quality depends on four major high-levelproperties of the software Correctness, internal characteristics, contextual characteristics and certaindescriptive properties. Each of these high-level properties of a software product, in turn, depends on several lower-level quality attributes. Dromey's hierarchical quality model is shown in Fig 13.2

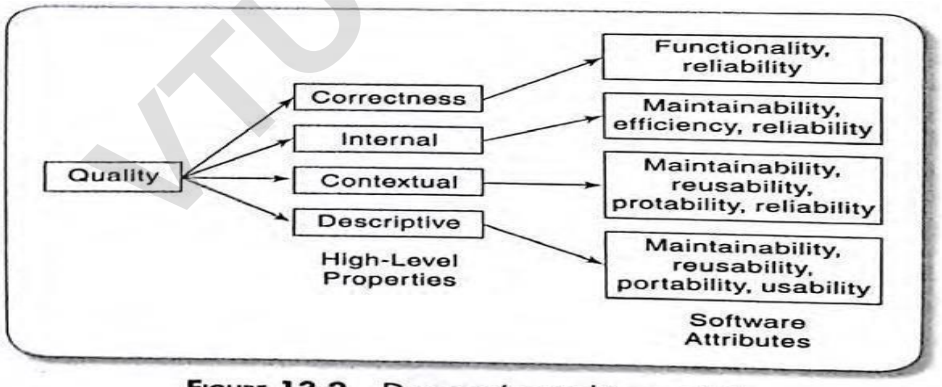


FIGURE 13.2 Dromey's quality model

3) Boehm's Model: Boehm's suggested that the quality of a software can be defined based on these high-level characteristics that are important for the users of the software. These three high-level characteristics are the following:

- ✓ **As-is -utility:** How well (easily, reliably and efficiently) can it be used?
- ✓ **Maintainability:** How easy is to understand, modify and then retest the software? **Portability:** How difficult would it be to make the software in a changed environment?

❖ Boehm's expressed these high-level product quality attributes in terms of several measurable product attributes. Boehm's hierarchical quality model is shown below figure.

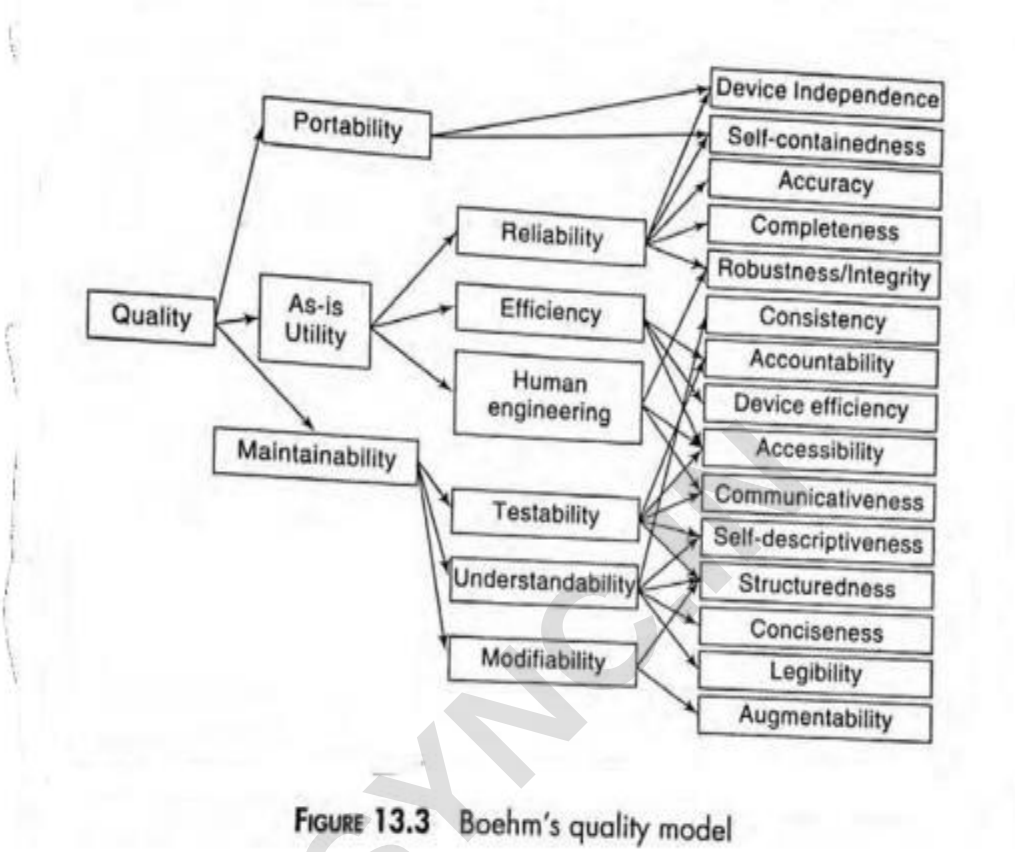


FIGURE 13.3 Boehm's quality model

VTUSYNC.IN

PRODUCT AND PROCESS METRICS

- Users assess the quality of a software product based on its external attributes, whereas during development, the developers assess the product's quality based on various internal attributes.
- The internal attributes may measure either some aspects of a product or of the development process (calledprocess metrics).

1. Product Metrics:

Purpose: Measure the characteristics of the software product being developed.

Examples:

- **Size Metrics**: Such as Lines of Code (LOC) and Function Points, which quantify the size orcomplexity of the software.
- **Effort Metrics**: Like Person-Months (PM), which measure the effort required to develop the software.
- **Time Metrics:** Such as the duration in months or other time units needed to complete thedevelopment.

2. Process Metrics:

Purpose: Measure the effectiveness and efficiency of the development process itself**.**

Examples:

- **Review Effectiveness:** Measures how thorough and effective code reviews are in finding defects.
- **Defect Metrics**: Average number of defects found per hour of inspection, average time taken to correct defects, and average number of failures detected during testing per line of code.
- **Productivity Metrics:** Measures the efficiency of the development team in terms of output per unit of effort or time.
- **Quality Metrics:** Such as the number of latent defects per line of code, which indicates the robustness of the software after development.

Differences:

- **Focus**: Product metrics focus on the characteristics of the software being built (size, effort, time), while process metrics focus on how well the development process is performing (effectiveness, efficiency, quality).
- **Use**: Product metrics are used to gauge the attributes of the final software product, aiding in planning, estimation, and evaluation. Process metrics help in assessing and improving the development process itself, aiming to enhance quality, efficiency, and productivity.
- **Application**: Product metrics are typically applied during and after development phases to assess the product's progress and quality. Process metrics are applied throughout the development lifecycle to monitor and improve the development process continuously.

❖ By employing both types of metrics effectively, software development teams can better manage projects,optimize processes, and deliver high-quality software products that meet user expectations.

VTUSYNC.IN

PRODUCT VERSUS PROCESS QUALITY MANAGEMENT

❖ In software development, managing quality can be approached from **two main perspectives**: **product quality management and process quality management**.

Product Quality Management

❖ Product quality management focuses **on evaluating and ensuring the quality of the software product itself**.This approach is typically more straight forward to implement and measure after the software has been developed.

Aspects:

1. Measurement Focus**:** Emphasizes metrics that assess the characteristics and attributes of the final software product, such as size (LOC, function points), reliability (defects found per LOC), performance(response time), and usability (user satisfaction ratings).

2. Evaluation Timing**:** Product quality metrics are often measured and evaluated after the software product has been completed or at significant milestones during development.

3. Benefits:

- **Provides clear benchmarks for evaluating the success of the software development project**.
- Facilitates comparisons with user requirements and industry standards.
- Helps in identifying areas for improvement in subsequent software versions or projects.

4. Challenges:

- **Predicting final product quality based on intermediate stages (like early code modules orprototypes) can be challenging.**
- Metrics may not always capture the full complexity or performance of the final integrated product.

Process Quality Management

- Process quality **management focuses on assessing and improving the quality of the development processes used to create the software. This approach aims to reduce errors and improve efficiency throughout the development lifecycle.**

Aspects:

1. Measurement Focus: Emphasizes metrics related to the development processes themselves, such as defect detection rates during **inspections, rework effort, productivity** (e.g., lines of code produced per hour**), and adherence to** defined standards and procedures.

2. Evaluation Timing: Process quality metrics are monitored **continuously throughout the development life cycle, from initial planning through to deployment and maintenance**.

3. Benefits:

- Helps in **identifying and correcting errors early in the development process**, reducing the cost andeffort of rework.
- Facilitates **continuous improvement of development practices**, leading to higher overall quality insoftware products.
- Provides **insights into the effectiveness of development methodologies** and practices used by theteam.

4. Challenges:

- **Requires consistent monitoring and analysis of metrics throughout the development lifecycle.**
- Effectiveness of process improvements may not always translate directly into improved product quality without careful management and integration.

Integration and Synergy

- While product and process quality management approaches have distinct focuses, they arecomplementary.
- Effective software development teams often integrate both approaches to achieve optimal results.
- By improving process quality, **teams can enhance product quality metrics, leading to morereliable, efficient, and user-friendly software products.**

VTUSYNC.IN

Software Project Estimation:

- Software Project Estimation is **the process of estimating various resources required for the completion of a project.**
- Estimation is the process of finding an **estimate, or approximation, which is a value that can be used for some purpose even if input data may be incomplete, uncertain, or unstable.**
- Effective software project estimation is one of the most challenging and important activities in software development.
- Proper project planning and control is not possible **without a reliable estimate**.
- The software industry doesn't estimate projects well and doesn't use estimates appropriately.

The four basic steps in software project estimation are:

1. **Estimate the size of the development product**. The size can be estimated by using either Lines of Code (LOC) or Function Points (FP).
2. **Estimate the effort in person-months or person-hours** (man-month or man-hour). Man-month is an estimate of personal resources required for the project.
3. **Estimate the schedule in calendar** days /months/ years based on total man-month required and manpower allocated to the project.
4. **Estimate the project cost in local currency**.

Observations on Estimation

- Estimation need not be a one-time task in a project. It can take organize during
  - ✓ **Acquiring a Project.**
  - ✓ **Planning the Project.**
  - ✓ **Execution of the Project as the need arises.**
- Project scope must be understood before the estimation **process begins**. It will be helpful to have historical Project Data.
- Project metrics can provide a historical perspective and valuable input for generation of quantitative estimates.
- Planning requires technical managers and the software team to make an initial commitment as it leads to responsibility and accountability.
- Experience can aid greatly.
- Use at least two estimation techniques to arrive at the estimates and reconcile the resulting values. Refer Decomposition Techniques in the next section to learn about reconciling estimates.
- Plans should be iterative and allow adjustments as time passes and more details are known.

VTUSYNC.IN

Decomposition Technique

- Is a **General Project Estimation Approach**
- Decomposition techniques take a **divide and conquer approach. Size, Effort and Cost estimation are performed in a stepwise manner by breaking down a Project into major Functions or related Software Engineering Activities.**

Step 1

- **Understand the scope of the software to be built.**

Step 2

- **Generate an estimate of the software size.**
  - Start with the statement of scope.
  - Decompose the software into functions that can each be estimated individually.
  - Calculate the size of each function.
  - Derive effort and cost estimates by applying the size values to your baseline productivity metrics.
  - Combine function estimates to produce an overall estimate for the entire project.

Step 3

- **Generate an estimate of the effort and cost. You can arrive at the effort and cost estimates by breaking down a project into related software engineering activities.**
  - Identify the sequence of activities that need to be performed for the project to be completed.
  - Divide activities into tasks that can be measured.
  - Estimate the effort (in person hours/days) required to complete each task.
  - Combine effort estimates of tasks of activity to produce an estimate for the activity.
  - Obtain cost units (i.e., cost/unit effort) for each activity from the database.
  - Compute the total effort and cost for each activity.
  - Combine effort and cost estimates for each activity to produce an overall effort and cost estimate for the entire project.

Step 4

- **Reconcile estimates: Compare the resulting values from Step 3 to those obtained from Step 2. If both sets of estimates agree, then your numbers are highly reliable. Otherwise, if widely divergent estimates occur conduct further investigation concerning whether**
  - The scope of the project is not adequately understood or has been misinterpreted.
  - The function and/or activity breakdown is not accurate.
  - Historical data used for the estimation techniques is inappropriate for the application, or obsolete, or has been misapplied.

Step 5

- **Determine the cause of divergence and then reconcile the estimates.**

VTUSYNC.IN

Empirical Estimation Technique

- Empirical estimation is a technique or model in which **empirically derived formulas are used for predicting the data that are a required and essential part of the software project planning step.**
- These techniques are usually **based on the data that is collected previously from a project and based on some guesses, prior experience with the development of similar types of projects, and assumptions.**
- It uses the size of the software to estimate the effort. In this technique, an educated guess of project parameters is made.
- These models are based on common sense.

VTUSYNC.IN